

<통신시스템 캡스톤 디자인1>

8주차 보고서

라즈베리파이와 FastAPI 서버를 활용한
실시간 등록·미등록 인물 판별 시스템 구현

부제:

- YOLOv8n, DeepFace ArcFace,
MySQL, Telegram·Email 알림 연동 -

-오석준-

체크리스트

DAY	주요 계획	목표	비고
3.31	Raspberry Pi OS 기반 환경 점검 SSH 원격 접속 구성 네트워크 및 IP 확인 Python/pip/venv 환경 구성, YOLO 설치 및 실행 가능성 검토	라즈베리파이 개발 환경 구축 완료 YOLO 실행 오류 (Illegal instruction) 확인	완료
4.7	PC 기반 FastAPI 서버 구축 기본 API(GET/POST) 작성 라즈베리파이↔서버 통신 확인	라즈베리파이와 PC 서버 간 통신 가능 상태 확보	완료
4.14	USB 웹캠(Logitech C920) 연결 카메라 인식 확인 OpenCV 기반 영상 캡처 및 이미지 저장	영상 입력 환경 구축 완료	완료
	라즈베리파이에서 캡처한 이미지를 PC 서버로 전송 HTTP POST 방식 업로드 기능 구현 라이브 스트리밍 기능 구현	이미지 전송 기능 구현	
5.8	PC 서버에서 YOLOv8n/v8s 실행 환경 구성 이미지 기반 객체 탐지 테스트	객체 탐지 기능 1차 구현	완료
5.19	등록 얼굴 데이터 수집, 얼굴 검출 및 특 정 추출 방식 설계, 등록 인물과 미등록 인물을 구분하기 위한 얼굴 인식 구조 설계, 결과 저장을 위한 DB 구조 설계	얼굴 인식 엔진 및 데 이터 저장 체계 확립	완료
	Opencv 기반 얼굴 검출과 특징 비교 기 능 구현, 등록 인물/미등록 인물 판별 테 스트, yolo 사람 탐지 결과와 얼굴 인식 결과를 결합한 판별하는 로직을 구현	등록 인물과 외부인 구 분 기능 구현, 객체 탐 지와 얼굴 인식을 결합 한 판별 구조 확인	
5.26	이벤트 정의 및 알람 기능 구현 침입 발생 시 로그/알람 생성	이벤트 기반 시스템 완성	완료
6.2	전체 시스템 통합 테스트 오류 수정 및 성능 개선	1학기 1차 완성본 도출	완료

목 차

1. 수행 개요
2. 전체 시스템 구성
 - 2.1 시스템 구성 요소
 - 2.2 전체 동작 흐름
3. 라즈베리파이 실시간 촬영 클라이언트 구현
 - 3.1 USB 웹캠 기반 이미지 캡처
 - 3.2 촬영 해상도 및 촬영 간격 설정
 - 3.3 HTTP POST 방식 이미지 전송
 - 3.4 서버 응답 결과 출력
4. FastAPI 기반 실시간 분석 서버 구현
 - 4.1 서버 실행 구조
 - 4.2 /health 서버 상태 확인 API
 - 4.3 /detect 이미지 분석 API
 - 4.4 원본 이미지, 결과 이미지 및 알림 파일 저장 구조
5. YOLOv8n 기반 사람 탐지 구현
 - 5.1 YOLOv8n과 YOLOv8s 비교 및 최종 모델 선정
6. DeepFace ArcFace 기반 얼굴 판별 개선
 - 6.1 RetinaFace detector 적용
 - 6.2 2단계 threshold 적용
7. 이벤트 분류 및 결과 저장 구조
 - 7.1 final_result와 event_type 매핑
 - 7.2 alert_log.txt 알림 로그 저장
8. 텔레그램 및 이메일 알림 시스템
 - 8.1 threading 기반 비동기 알림
9. 실시간 통합 실험 결과

- 9.1 등록 인물 판별 결과
 - 9.2 미등록 인물 판별 결과
 - 9.3 다중 인물 동시 판별 결과
 - 9.4 처리 속도 분석
 - 9.5 해상도 비교 결과
 - 9.6 실험 결과 종합
- 10. 저번 주차 대비 보완 사항
 - 10.1 .env 기반 민감 정보 분리
 - 10.2 .gitignore를 통한 민감 파일 및 결과 파일 제외
- 11. 결론

1. 수행 개요

이번 주차에서는 기존에 구현한 YOLOv8n 기반 사람 탐지, DeepFace ArcFace 기반 얼굴 판별, MySQL 저장, 텔레그램 및 이메일 알림 기능을 라즈베리파이 실시간 촬영 환경과 연동하였다. 라즈베리파이는 USB 웹캠을 이용하여 이미지를 주기적으로 촬영하고, PC에서 실행 중인 FastAPI 서버의 /detect API로 이미지를 전송하도록 구성하였다.

PC 서버는 수신한 이미지를 저장한 뒤 YOLOv8n 모델을 이용하여 사람 객체를 탐지하고, 탐지된 사람 영역을 crop하여 DeepFace ArcFace 모델로 등록 얼굴과 비교하였다. 비교 결과에 따라 Registered, Unknown, Unidentified, NoPerson, CropFailed 등으로 최종 판별하고, 판별 결과를 MySQL DB에 저장하였다. 또한 보안상 확인이 필요한 Unknown, Unidentified, CropFailed 이벤트에 대해서는 텔레그램 및 이메일 알림을 전송하도록 구현하였다.

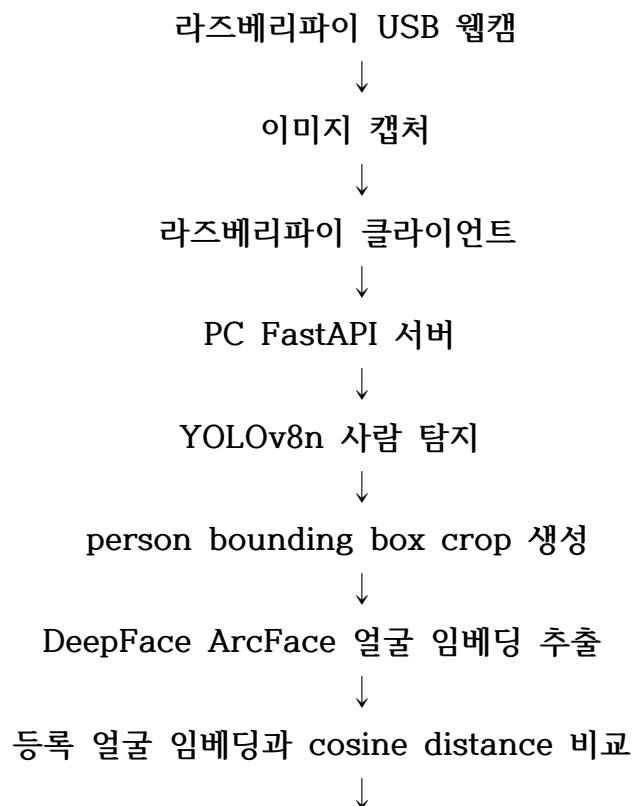
특히 이번 주차에서는 실시간 테스트 결과를 바탕으로 등록 인물과 미등록 인물의 distance 분포를 비교하고, 기존 단일 threshold 방식보다 안정적인 2단계 threshold 구조를 적용하였다. 이를 통해 확실한 등록 인물은 정상 접근으로 처리하고, 미등록 가능성이 높거나 식별이 애매한 상황은 관리자 확인 대상으로 분류하도록 개선하였다.

2. 전체 시스템 구성

2.1 시스템 구성 요소

구성 요소	역할
Raspberry Pi	USB 웹캠으로 이미지 촬영 및 PC 서버 전송
USB Webcam	실시간 실내 이미지 입력 장치
FastAPI Server	이미지 수신, 분석, 결과 반환
YOLOv8n	사람 객체 탐지
DeepFace ArcFace	등록 얼굴과 입력 얼굴의 임베딩 비교
RetinaFace	얼굴 검출 detector backend
MySQL	탐지 결과 및 이벤트 정보 저장
Telegram Bot	미등록/식별 실패 이벤트 실시간 알림
Gmail SMTP	이벤트 상세 정보 이메일 전송
.env 파일	DB, Telegram, Email 등 민감 정보 관리

2.2 전체 동작 흐름



Registered / Unknown / Unidentified 판별



MySQL 저장 + 결과 이미지 저장 + Telegram/Email 알림

3. 라즈베리파이 실시간 촬영 클라이언트 구현

3.1 USB 웹캠 기반 이미지 캡처

라즈베리파이 클라이언트는 OpenCV를 이용하여 USB 웹캠을 제어하였다. cv2.VideoCapture()를 통해 카메라 장치를 열고, CAP_PROP_FRAME_WIDTH, CAP_PROP_FRAME_HEIGHT 설정을 통해 촬영 해상도를 지정하였다. 이후 실제 적용된 해상도를 출력하여 설정값이 카메라에 정상적으로 반영되었는지 확인하였다.

```
cap = cv2.VideoCapture(CAMERA_INDEX)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, FRAME_WIDTH)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, FRAME_HEIGHT)
actual_w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
actual_h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
print(f"[카메라] 실제 해상도: {actual_w}x{actual_h}")
```

3.2 촬영 해상도 및 촬영 간격 설정

라즈베리파이 클라이언트는 .env 파일을 통해 서버 IP, 포트, 카메라 번호, 촬영 해상도, 촬영 간격, 요청 제한 시간 등을 설정하도록 구성하였다.

```
SERVER_IP=192.168.1.12
SERVER_PORT=8000
CAMERA_INDEX=0 #/dev/video0 USB 웹캠 사용
FRAME_WIDTH=1920
FRAME_HEIGHT=1080
```

```
CAPTURE_INTERVAL=3.0 #촬영 간격
SAVE_LOCAL=False #라즈베리파이 내부 저장 없이 서버 전송 중심
REQUEST_TIMEOUT=15 #서버 응답 지연 시 타임아웃 처리
```

본 실험에서는 라즈베리파이 촬영 간격을 CAPTURE_INTERVAL=3.0으로 설정하였다. 따라서 라즈베리파이는 약 3초 간격으로 프레임을 캡처하고, 촬영된 이미지를 PC FastAPI 서버의 /detect API로 전송한다. 다만 실제 처리 주기는 촬영 간격뿐만 아니라 서버 분석 시간의 영향을 받는다. 사람이 감지되지 않은 경우에는 서버 응답이 빠르게 반환되어 3초 주기에 가깝게 동작하였으나, 사람이 감지되어 DeepFace 기반 얼굴 검출 및 임베딩 비교가 수행되는 경우에는 약 6~10초의 처리 시간이 소요되었다.

3.3 HTTP POST 방식 이미지 전송

촬영된 프레임은 OpenCV의 cv2.imencode()를 이용하여 JPEG 형식으로 변환하였다. 이후 Python requests 라이브러리를 사용하여 FastAPI 서버의 /detect API로 HTTP POST 요청을 전송하였다. 서버로부터 반환된 JSON 응답에는 최종 판별 결과, YOLO confidence, DeepFace distance, 이벤트 종류, 알림 필요 여부 등이 포함된다.

```
success, encoded = cv2.imencode(".jpg", frame,
[ cv2.IMWRITE_JPEG_QUALITY, 85])
files = {
    "file": (filename, encoded.tobytes(), "image/jpeg")
}

r = requests.post(
    f"{SERVER_URL}/detect",
    files=files,
    timeout=REQUEST_TIMEOUT
)
```

3.4 서버 응답 결과 출력

라즈베리파이 클라이언트는 촬영한 이미지를 PC FastAPI 서버의 /detect API로 전송한 뒤, 서버로부터 반환된 JSON 응답을 터미널에 출력하였다. 출력 결과에는 이미지 파일명, 최종 판별 결과, YOLO confidence, DeepFace distance, event_type, 서버 처리 시간이 포함된다. 예를 들어 Unknown으로 판별된 경우 UNKNOWN_PERSON_DETECTED 이벤트가 출력되며, 이는 미등록 인물이 감지되었음을 의미한다

```
[0043] *** ALERT *** pi_20260601_210813_825.jpg -> Unknown (conf=0.95, dist=0.551) event=UNKNOWN_PERSON_DETECTED | 7131.33ms
[0044] *** ALERT *** pi_20260601_210821_898.jpg -> Unknown (conf=0.94, dist=0.577) event=UNKNOWN_PERSON_DETECTED | 7012.86ms
[0045] *** ALERT *** pi_20260601_210828_264.jpg -> Unknown (conf=0.94, dist=0.517) event=UNKNOWN_PERSON_DETECTED | 7433.15ms
[0046] *** ALERT *** pi_20260601_210835_942.jpg -> Unknown (conf=0.94, dist=0.562) event=UNKNOWN_PERSON_DETECTED | 7881.84ms
[0047] *** ALERT *** pi_20260601_210843_931.jpg -> Unknown (conf=0.95, dist=0.557) event=UNKNOWN_PERSON_DETECTED | 7437.16ms
```

출처: [라즈베리파이 post](#)

4. FastAPI 기반 실시간 분석 서버 구현

4.1 서버 실행 구조

이번 최종 서버는 FastAPI 기반으로 구현되었으며, 라즈베리파이로부터 이미지를 POST로 수신하고 YOLO와 DeepFace를 이용해 등록·미등록 인물을 판별한다. 또한 MySQL 저장, Telegram 알림, Email 알림, threading 기반 비동기 알림, .env 환경변수 분리 기능을 포함한다. 서버 코드에는 .env 로드, DB 설정, Telegram 설정, Gmail SMTP 설정, 기능 ON/OFF 토글, 얼굴 인식 threshold, YOLO 모델 로드, FastAPI API 정의가 포함되어 있다.

4.2 /health 서버 상태 확인 API

/health API는 서버가 정상적으로 실행 중인지, 등록 얼굴 임베딩이 몇 개 로드되었는지, 텔레그램·이메일·DB 기능이 활성화되어 있는지 확인하기 위해 사용하였다.

```

@app.get("/health")
def health():
    return {
        "status": "ok",
        "registered_faces": {d: len(EMBEDDINGS_CACHE[d]) for d in
DETECTOR_BACKENDS},
        "total_registered": total_registered,
        "register_threshold": REGISTER_THRESHOLD,
        "unknown_threshold": UNKNOWN_THRESHOLD,
        "early_exit_distance": EARLY_EXIT_DISTANCE,

        "detectors": DETECTOR_BACKENDS,
        "telegram_enabled": ENABLE_TELEGRAM_ALERT,
        "email_enabled": ENABLE_EMAIL_ALERT,
        "db_enabled": ENABLE_DB_LOGGING,

    }

```



```

{
  "status": "ok",
  "registered_faces": {
    "retinaface": 21
  },
  "total_registered": 21,
  "register_threshold": 0.3,
  "unknown_threshold": 0.47,
  "early_exit_distance": -1,
  "detectors": [
    "retinaface"
  ],
  "telegram_enabled": true,
  "email_enabled": true,
  "db_enabled": true,
}

```

※등록인물의 저장된 이미지가 21개인 것을 알 수 있음.

4.3 /detect 이미지 분석 API

/detect API는 라즈베리파이에서 전송한 이미지를 수신하고, 저장, YOLO 탐지, DeepFace 비교, 결과 이미지 저장, DB 저장, 알림 전송까지 수행하는 핵심 API이다.

```
@app.post("/detect")
```

```
async def detect(file: UploadFile = File(...)):
```

```
    start = time.time()
```

```
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")[:-3]
```

```
    img_name = f"{timestamp}_{file.filename}"
```

```
    img_path = UPLOAD_DIR / img_name
```

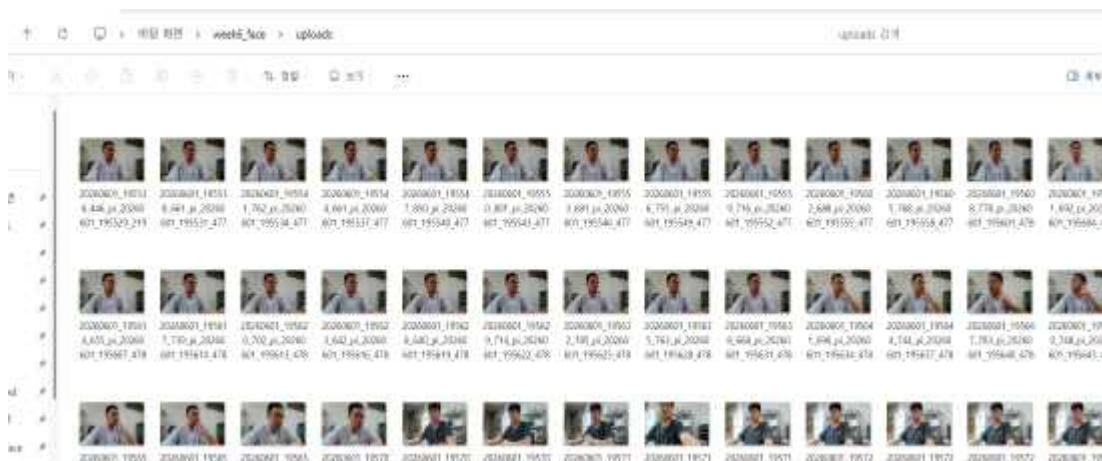
```
    contents = await file.read()
```

```
    with open(img_path, "wb") as f:
```

```
        f.write(contents)
```

```
    image = cv2.imread(str(img_path))
```

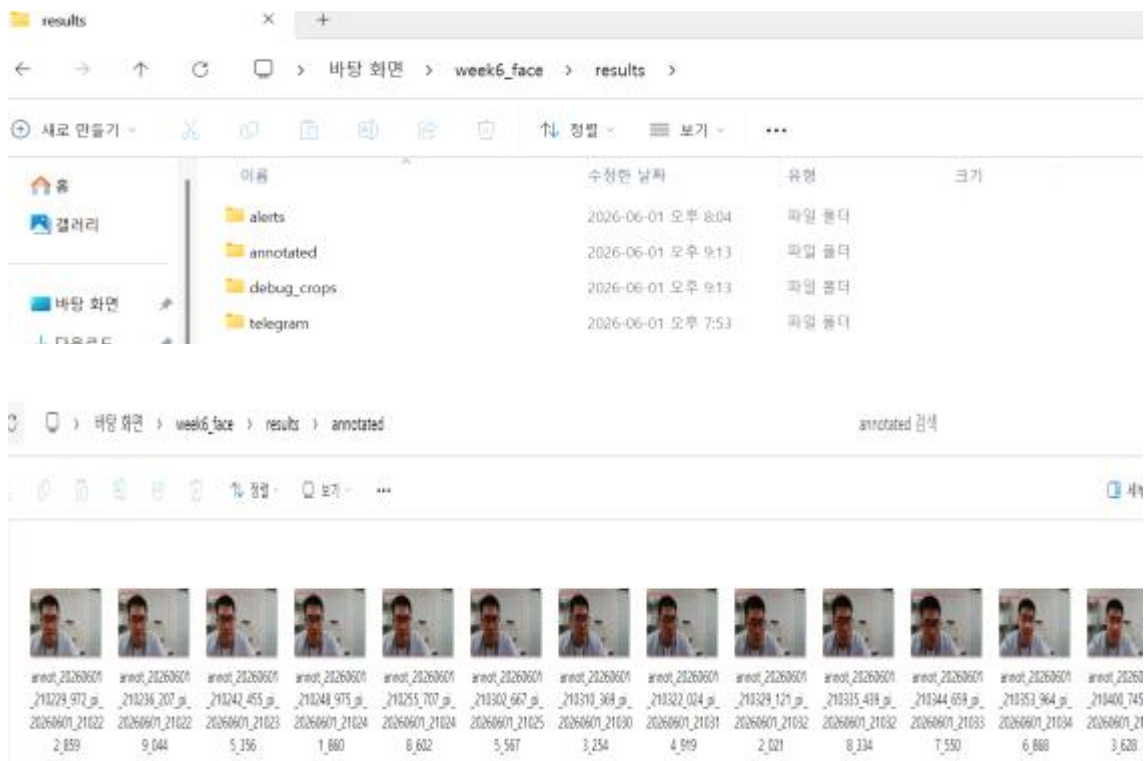
/detect API는 라즈베리파이에서 전송한 이미지 파일을 수신한 뒤, 서버의 uploads 폴더에 저장한다. 이후 OpenCV로 이미지를 다시 로드하여 YOLOv8n 사람 탐지 단계로 전달한다. 이 구조를 통해 원본 이미지를 보존하면서 분석 결과 이미지와 비교할 수 있도록 하였다.



4.4 원본 이미지, 결과 이미지 및 알림 파일 저장 구조

폴더	역할
uploads/	라즈베리파이에서 전송한 원본 이미지 저장
results/alerts/	알림 발생 내역을 기록한 로그 파일 저장
results/annotated/	YOLO box와 판별 결과가 표시된 결과 이미지 저장
results/debug_crops/	얼굴 비교에 사용된 crop 이미지 저장
results/telegram/	텔레그램 전송용 이미지 또는 알림 전송에 사용되는 이미지 저장

서버는 라즈베리파이에서 수신한 원본 이미지를 uploads 폴더에 저장하고, YOLO 및 DeepFace 판별 결과가 표시된 이미지는 results/annotated 폴더에 저장하였다. 또한 얼굴 비교 과정에서 생성된 crop 이미지는 results/debug_crops 폴더에 저장하여, 어떤 영역이 얼굴 비교에 사용되었는지 추후 확인할 수 있도록 하였다.



출처: [fastapi restfulapi 각종 사용법](#)

5. YOLOv8n 기반 사람 탐지 구현

5.1 YOLOv8n과 YOLOv8s 비교

이전 실험에서는 YOLOv8n과 YOLOv8s 모델을 비교하여 사람 탐지 성능과 처리 속도를 확인하였다. YOLOv8s는 YOLOv8n보다 모델 규모가 크기 때문에 탐지 안정성이 다소 높을 수 있지만, 실시간 처리 환경에서는 처리 시간이 증가할 가능성이 있다. 반면 YOLOv8n은 더 작은 모델로, 비교적 빠른 추론 속도를 제공하므로 라즈베리파이에서 전송되는 이미지를 PC 서버에서 반복적으로 처리하는 구조에 적합하다.

비교 항목	YOLOv8n	YOLOv8s	최종 판단
모델 크기	작음	상대적으로 큼	YOLOv8n 유리
처리 속도	빠름	상대적으로 느림	YOLOv8n 유리
사람 탐지 성능	실내 환경에서 충분	더 안정적일 수 있음	둘 다 사용 가능
서버 부하	낮음	높음	YOLOv8n 유리
DeepFace 연동	person crop 생성 가능	person crop 생성 가능	YOLOv8n으로 충분
최종 적용	적용	비교 후 제외	YOLOv8n 선정

YOLOv8s는 YOLOv8n보다 탐지 안정성이 높을 수 있으나, 본 시스템에서는 사람 탐지 이후 DeepFace 얼굴 판별이 추가로 수행되기 때문에 전체 처리 시간이 중요한 요소로 작용하였다. YOLOv8n은 경량 모델이면서도 실내 환경에서 사람 탐지에는 충분한 성능을 보였고, DeepFace 단계로 전달할 person crop을 생성하는 데 문제가 없었다. 따라서 최종 실시간 시스템에서는 처리 속도와 시스템 부담을 고려하여 YOLOv8n을 사용하였다.

6. DeepFace ArcFace 기반 얼굴 판별 개선

6.1 RetinaFace detector 적용

최종 코드에서는 detector backend를 retinaface로 제한하였다.

```
DETECTOR_BACKENDS = ["retinaface"]
```

얼굴 검출 backend는 RetinaFace를 사용하였다. OpenCV detector는 처리 속도가 빠르지만 얼굴 검출 정확도가 낮거나 오탐 가능성이 존재하여, 최종 실시간 판별에서는 RetinaFace만 사용하였다. 이를 통해 처리 속도는 다소 증가할 수 있으나, 얼굴 검출 및 임베딩 추출의 안정성을 우선하였다.

6.2 2단계 threshold 적용

이전

```
REGISTER_THRESHOLD = 0.35  
UNKNOWN_THRESHOLD = 0.50
```

최종 서버

```
REGISTER_THRESHOLD = 0.30  
UNKNOWN_THRESHOLD = 0.47  
EARLY_EXIT_DISTANCE = -1
```

*Early Exit는 매우 낮은 distance가 측정될 경우 나머지 비교를 생략하기 위한 속도 개선 방식이다. 그러나 본 실험에서는 안정적인 판별을 위해 EARLY_EXIT_DISTANCE=-1로 설정하여 비활성화하였다.

판별 기준

```
distance < 0.30           → Registered  
0.30 ≤ distance < 0.47   → Unidentified  
distance ≥ 0.47           → Unknown
```

```
if best["distance"] is None:
```

```
    best["final_result"] = "Unidentified"
```

```

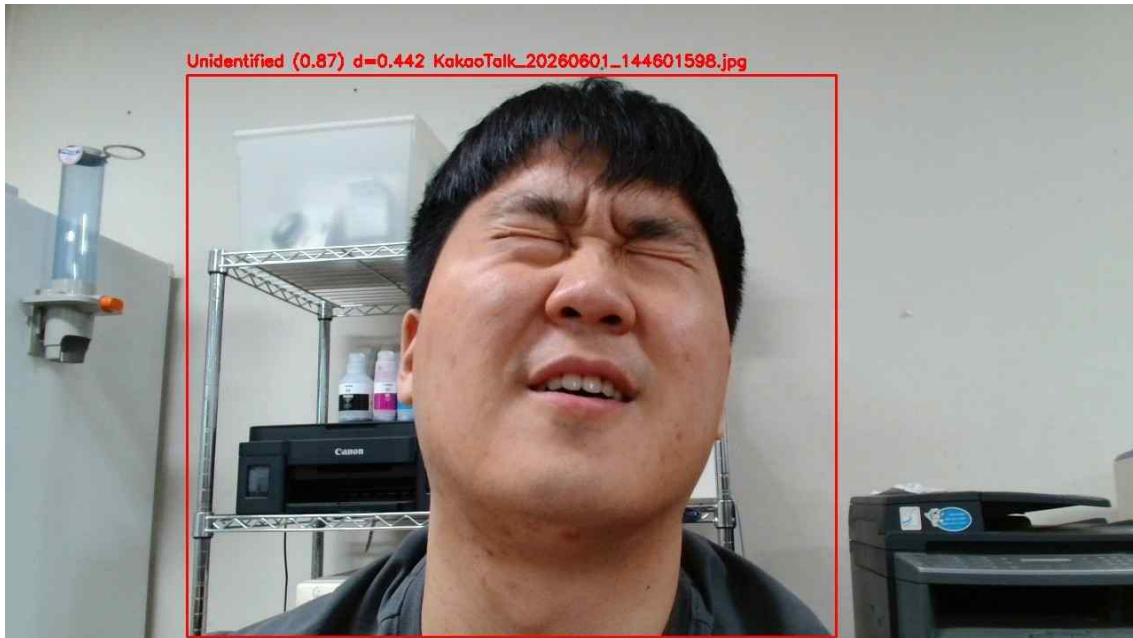
elif best["distance"] < REGISTER_THRESHOLD:
    best["final_result"] = "Registered"
elif best["distance"] >= UNKNOWN_THRESHOLD:
    best["final_result"] = "Unknown"
else:
    best["final_result"] = "Unidentified"

```

실시간 테스트 과정에서 등록 인물과 미등록 인물의 distance 값을 비교한 결과, 등록 인물의 일반적인 정면 얼굴은 약 0.16~0.27 범위로 측정되었다. 반면 미등록 인물은 대부분 0.50 이상으로 측정되었다. 그러나 등록 인물이 얼굴을 찡그리거나 표정 변화가 큰 경우, 그리고 미등록 인물이 정면 조건에서 촬영된 경우에는 0.30~0.47 사이의 애매한 distance 값이 나타났다.

실험 대상	조건	distance	기존 단일 기준 해석 가능성	최종 판별
등록 인물	정면, 일반 표정	0.10~0.27	등록 인물	Registered
등록 인물	표정 변화	0.30~0.44	애매함	Unidentified
미등록 인물	일반 조건	0.50~0.69	미등록 인물	Unknown
미등록 인물	정면, 안경 미착용, 유사 조명	0.42~0.46	애매함	Unidentified
얼굴 검출 실패	얼굴이 작거나 가려짐	None	비교 불가	Unidentified

[쫄그린 표정으로 인한 distance = 0.44]



따라서 본 시스템은 단일 threshold로 등록 여부를 결정하지 않고, 2단계 threshold를 적용하였다. distance가 0.30 미만인 경우에만 확실한 등록 인물로 판단하여 Registered로 분류하고, 0.30 이상 0.47 미만인 경우에는 등록 여부를 확정하기 어려운 상태로 보아 Unidentified로 분류하였다. Unidentified는 단순히 얼굴이 없다는 의미가 아니라, 얼굴 검출 실패 또는 등록 여부가 애매한 경우를 포함한다. 즉, 관리자 확인 대상으로 보는 것이다. 이는 미등록 인물이 등록 인물로 오분류되는 위험을 줄이기 위한 판별 방식이다. distance가 0.47 이상인 경우에는 미등록 가능성이 높다고 판단하여 Unknown으로 분류하였다.

7. 이벤트 분류 및 결과 저장 구조

7.1 final_result와 event_type 매핑

단순히 얼굴 판별 결과만 출력하는 것이 아니라, 시스템 동작 관점에서 이벤트를 분류하기 위해 `get_event_info()` 함수를 구현하였습니다. Registered는 정상 접근으로 판단하여 `NORMAL_ACCESS` 이벤트로 분류하고, 알림은 발생시키지 않는다. 반면 Unknown은 `UNKNOWN_PERSON_DETECTED`, Unidentified는 `FACE_NOT_IDENTIFIED`, CropFailed는 `CROP_FAILED` 이벤트로 분류하여 텔레그램 및 이메일 알림 대상이 되도록 하였다.

```
def get_event_info(final_result):
    mapping = {
        "Registered": (
            "NORMAL_ACCESS",
            False,
            "Registered person detected."
        ),
        "Unknown": (
            "UNKNOWN_PERSON_DETECTED",
            True,
            "Alert: Unknown person detected."
        ),
        "Unidentified": (
            "FACE_NOT_IDENTIFIED",
            True,
            "Alert: Face could not be identified or result is
ambiguous."
        ),
        "NoPerson": (
            "NO_PERSON",
            False,
```

```

        "No person detected."
    ),
    "CropFailed": (
        "CROP_FAILED",
        True,
        "Alert: Crop failed."
    ),
}

```

final_result	event_type	alert_required	의미
Registered	NORMAL_ACCE SS	False	등록 인물 정상 접근
Unknown	UNKNOWN_PER SON_DETECTE D	True	미등록 인물 감지
Unidentified	FACE_NOT_IDE NTIFIED	True	얼굴 식별 실패 또는 애매한 결과
NoPerson	NO_PERSON	False	사람 미탐지
CropFailed	CROP_FAILED	True	사람 탐지 후 crop 실패

7.2 alert_log.txt 알림 로그 저장

텔레그램 및 이메일 알림과 별도로, 알림 발생 내역은 results/alerts/alert_log.txt 파일에 저장하였다. 이는 알림 메시지가 삭제되거나 확인이 어려운 경우에도 이벤트 발생 이력을 텍스트 형태로 추적할 수 있도록 하기 위한 보조 역할을 한다.

```

def save_alert_log(event_info, row):
    if not event_info["alert_required"]:
        return

```

```

log_file = ALERT_DIR / "alert_log.txt"
with open(log_file, "a", encoding="utf-8") as f:
    f.write(
        f"[{row['detected_at']}] "
        f"{event_info['event_type']} | "
        f"{row['image_name']} | "
        f"result={row['final_result']} | "
        f"dist={row['distance']} | "
        f"matched={row['matched']} | "
        f"{event_info['alert_message']}\n"
    )

```

```

Face could not be identified or result is ambiguous.
[2026-06-01 20:05:48] UNKNOWN_PERSON_DETECTED | 20260601_200539_428_pi_20260601_200532_253.jpg | result=Unknown | dist=0.5221753716468811 | matched=register6.jpg |
Alert: Unknown person detected.
[2026-06-01 20:05:57] UNKNOWN_PERSON_DETECTED | 20260601_200548_730_pi_20260601_200541_571.jpg | result=Unknown | dist=0.6936885118484497 | matched=register20.jpg |
Alert: Unknown person detected.
[2026-06-01 20:06:24] UNKNOWN_PERSON_DETECTED | 20260601_200615_547_pi_20260601_200608_395.jpg | result=Unknown | dist=0.5049853324890137 | matched=register15.jpg |
Alert: Unknown person detected.
[2026-06-01 20:06:33] FACE_NOT_IDENTIFIED | 20260601_200624_461_pi_20260601_200617_309.jpg | result=Unidentified | dist=0.3794078230857849 | matched=register15.jpg | Alert:
Face could not be identified or result is ambiguous.

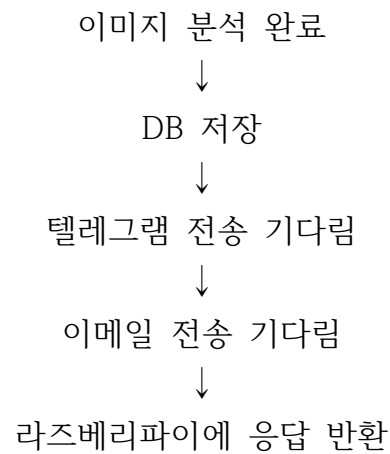
```

또한 실시간 분석 결과는 MySQL detection_log 테이블에 저장하였다. 저장 항목에는 이미지명, 사람 번호, YOLO confidence, 최종 판별 결과, DeepFace distance, detector, crop type, event_type, alert_required, annotated image path, 탐지 시간 등이 포함된다. 이를 통해 텔레그램 또는 이메일 알림 외에도 사후 분석과 결과 검증이 가능하도록 하였다.

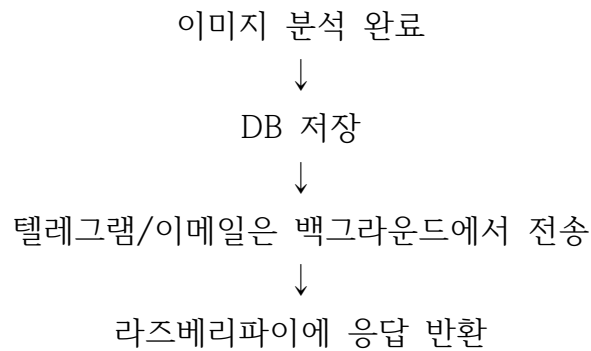
8. 텔레그램 및 이메일 알림 시스템

8.1 threading 기반 비동기 알림

동기 방식이면:



근데 threading을 쓰면:



if ENABLE_TELEGRAM_ALERT and TELEGRAM_BOT_TOKEN and TELEGRAM_CHAT_ID:

```
threading.Thread(  
    target=_send_telegram,  
    args=(event_info, row, image_path),
```

```

        daemon=True
    ).start()
    if ENABLE_EMAIL_ALERT and EMAIL_SENDER and
EMAIL_APP_PASSWORD and EMAIL_RECEIVER:
        threading.Thread(
            target=_send_email,
            args=(event_info, row, image_path),
            daemon=True
        ).start()

```

텔레그램과 이메일 전송은 네트워크 요청이 포함되어 있어 서버 응답 지연의 원인이 될 수 있다. 이를 줄이기 위해 최종 서버에서는 알림 전송을 별도의 thread로 실행하도록 구성하였다. 이를 통해 이미지 분석 결과는 먼저 반환하고, 알림 전송은 백그라운드에서 수행되도록 하여 실시간 처리 흐름의 지연을 줄이고자 하였다.

출처:[threading 방식](#)

2. 알림 메시지 시간 기능 추가

알림 메시지에는 탐지 시간이 함께 포함되도록 하여, 미등록 인물 또는 식별 실패 이벤트가 발생한 시점을 명확히 확인할 수 있도록 하였다. 이는 사후 로그 분석과 상황 확인에 필요한 기록성을 높이기 위한 보완 사항이다.

```

caption = (
    f"{{event_info['event_type']}}\n"
    f"{{event_info['alert_message']}}\n\n"
    f"이미지: {row['image_name']}\n"
    f"결과: {row['final_result']}\n"
    f"distance: {row['distance']}\n"
    f"matched: {row['matched']}\n"
    f"detector: {row['detector']}\n"
    f"crop: {row['crop_type']}\n"
    f"시간: {row['detected_at']}"
)

```

9. 실시간 통합 실험 결과

9.1 등록 인물 판별 결과

실험 조건	distance 범위	최종 판별	해석
등록 인물 정면	약 0.10~0.27	Registered	정상 등록 인물
등록 인물 표정 변화	약 0.30~0.44	Unidentified	표정 변화로 인한 애매한 결과
등록 인물 얼굴 식별 실패	None	Unidentified	얼굴 임베딩 추출 실패

등록 인물 실험에서는 일반적인 정면 상태에서 distance가 약 0.10~0.27 범위로 측정되어 Registered로 정상 판별되었다. 이 경우 event_type은 NORMAL_ACCESS로 매핑되며, 정상 접근 상황이므로 텔레그램 및 이메일 알림은 발생하지 않고 DB에만 기록되었다. 반면 얼굴을 찡그리거나 표정 변화가 큰 경우에는 distance가 증가하여 Unidentified로 분류되는 사례가 발생하였다.

정상 판별



안경으로 인한 unidentified



[FACE_NOT_IDENTIFIED]

Alert: Face could not be identified or result is ambiguous.

이미지: 20260601_210736_802_pi_20260601_210729_692.jpg

결과: Unidentified

distance: 0.4470473527908325

matched: register12.jpg

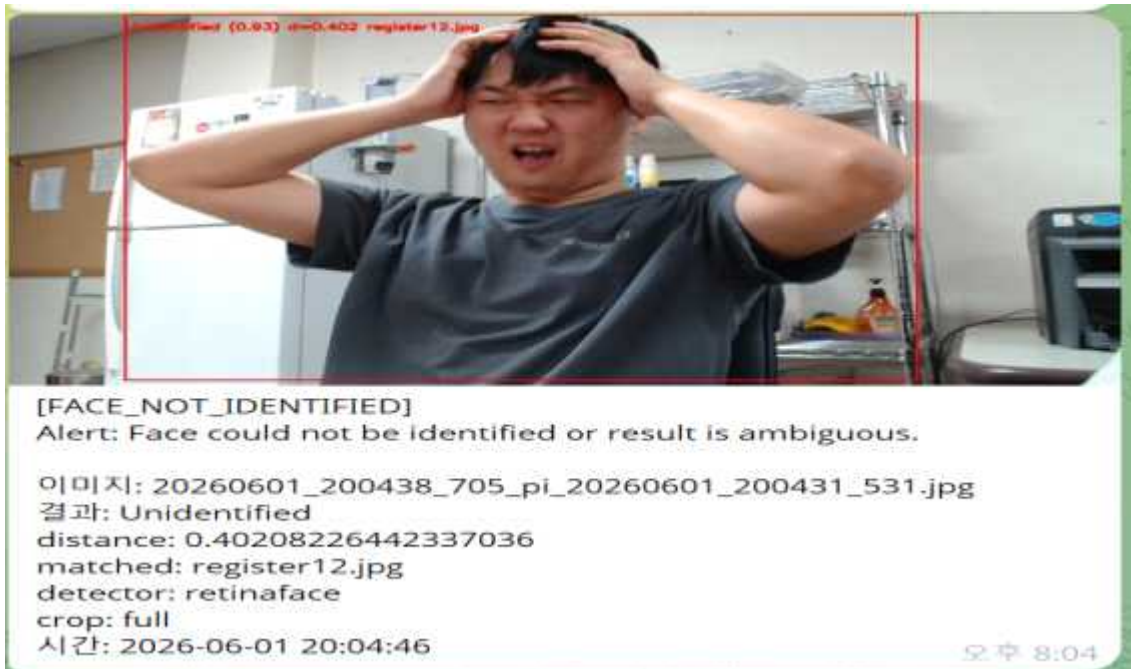
detector: retinaface

crop: full

시간: 2026-06-01 21:07:47

오후 9:07

짱그림으로 인한 unidentified



얼굴 가림으로 인한 unidentified



9.2 미등록 인물 판별 결과

실험 조건	distance 범위	최종 판별	해석
미등록 인물 일반 조건	약 0.50 이상	Unknown	미등록 인물로 판별
미등록 인물 정면 / 유사 조명	약 0.36~0.46	Unidentified	등록 여부 애매, 관리자 확인

미등록 인물 실험에서는 대부분 distance가 0.47 이상으로 측정되어 Unknown으로 분류되었다. Unknown은 UNKNOWN_PERSON_DETECTED 이벤트로 매핑되며, 보안상 확인이 필요한 상황으로 판단하여 텔레그램 및 이메일 알림을 전송하였다. 일부 미등록 인물은 안경 착용 여부, 정면 촬영, 조명 조건 등에 따라 0.30~0.47 사이의 애매한 distance 값을 보였으나, 이 경우에도 Registered로 처리하지 않고 Unidentified로 분류하였다.

미등록 인물 unknown 판별





[UNKNOWN_PERSON_DETECTED]
Alert: Unknown person detected.

이미지: 20260601_203223_788_pi_20260601_203216_663.jpg

결과: Unknown

distance: 0.5886645317077637

matched: register7.jpg

detector: retinaface

crop: full

시간: 2026-06-01 20:32:32

오후 8:32



[UNKNOWN_PERSON_DETECTED]
Alert: Unknown person detected.

이미지: 20260601_152622_515_pi_20260601_152615_555.jpg

결과: Unknown

distance: 0.7726603150367737

matched: KakaoTalk_20260601_144606695.jpg

detector: retinaface

crop: full

시간: 2026-06-01 15:26:28

오후 3:26



[UNKNOWN_PERSON_DETECTED]
Alert: Unknown person detected.

이미지: 20260601_152939_410_pi_20260601_152927_900.jpg

결과: Unknown

distance: 0.5368162393569946

matched: KakaoTalk_20260601_144601598.jpg

detector: retinaface

crop: full

시간: 2026-06-01 15:29:47

오후 3:29

[Capstone Alert] UNKNOWN_PERSON_DETECTED 받은편지함 ✕



lose84681221@gmail.com

나에게 ▾

침입 감지 시스템 알림

[이벤트 정보]

이벤트 종류 : UNKNOWN_PERSON_DETECTED

알림 메시지 : Alert: Unknown person detected.

최종 판별 결과 : Unknown

[탐지 정보]

이미지명 : 20260601_211348_757_pi_20260601_211341_655.jpg

YOLO 신뢰도 : 0.928911030292511

DeepFace 거리 : 0.5329159498214722

Detector : retinaface

Crop Type : full

매칭 이미지 : register15.jpg

탐지 시간 : 2026-06-01 21:13:56

첨부파일 1개 • Gmail에서 검사할 ① Drive에 추가



미등록 인물 unidentified



[FACE_NOT_IDENTIFIED]

Alert: Face could not be identified or result is ambiguous.

이미지: 20260601_201226_665_pi_20260601_201219_500.jpg

결과: Unidentified

distance: 0.3657483458518982

matched: register9.jpg

detector: retinaface

crop: full

시간: 2026-06-01 20:12:35

오후 8:12



[FACE_NOT_IDENTIFIED]

Alert: Face could not be identified or result is ambiguous.

이미지: 20260601_201217_845_pi_20260601_201210_690.jpg

결과: Unidentified

distance: 0.4179113507270813

matched: register9.jpg

detector: retinaface

crop: full

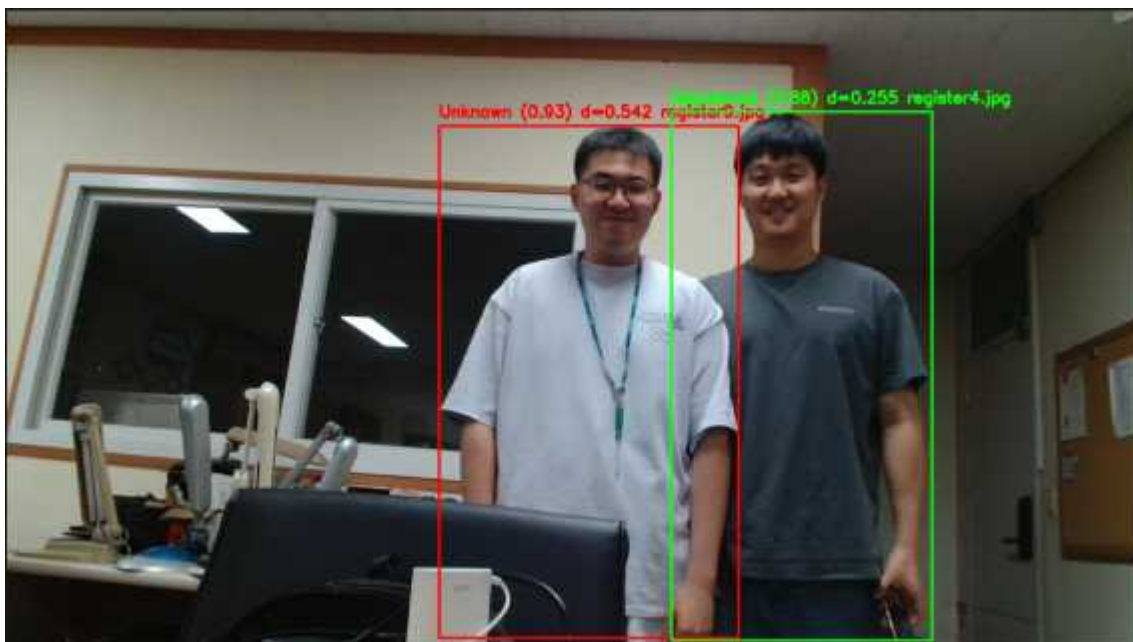
시간: 2026-06-01 20:12:26

오후 8:12

9.3 다중 인물 동시 판별 결과

한 프레임 안에 등록 인물과 미등록 인물이 동시에 존재하는 상황에서 시스템이 각 인물을 개별적으로 탐지하고 판별할 수 있는지 확인하였다. 동일한 이미지 안에서도 한 사람은 Registered, 다른 사람은 Unknown 또는 Unidentified로 분류될 수 있다.





다중 인물 실험 결과, 한 장의 이미지 안에 등록 인물과 미등록 인물이 동시에 존재하더라도 시스템은 각 인물을 독립적으로 탐지하고 판별할 수 있었다. 등록 인물은 대부분 distance가 0.30 미만으로 측정되어 Registered로 분류되었으며, 미등록 인물은 distance가 0.47 이상인 경우 Unknown으로 분류되었다. 또한 일부 미등록 인물은 정면 촬영, 조명, 얼굴 유사도 등의 영향으로 0.30~0.47 사이의 distance 값을 보여 Unidentified로 처리되었다.

이 결과는 본 시스템이 단일 인물 판별뿐만 아니라 실제 환경에서 발생할 수 있는 다중 인물 상황에서도 개별 인물 단위의 판별이 가능함을 보여준다. 특히 등록 인물과 미등록 인물이 동시에 등장한 경우에도 등록 인물은 정상 접근으로 처리하고, 미등록 인물 또는 애매한 인물은 알림 대상으로 분류할 수 있었다.

9.4 처리 조건별 서버 처리 시간 비교

처리 조건	주요 수행 단계	처리 시간	분석
사람 미탐지	YOLO 탐지만 수행	약 0.15~0.65초	DeepFace 단계 생략으로 빠름
등록 인물 탐지	YOLO + DeepFace + DB 저장	약 6~10초	얼굴 임베딩 비교로 시간 증가
미등록 인물 탐지	YOLO + DeepFace + 알림 처리	약 7~10초	알림 대상 이벤트 처리 포함

사람이 탐지되지 않은 경우에는 YOLO 사람 탐지만 수행되므로 약 0.15~0.65초 수준의 짧은 처리 시간이 나타났다. 반면 사람이 탐지되어 DeepFace 기반 얼굴 검출 및 ArcFace 임베딩 비교가 수행된 경우에는 약 6~10초의 처리 시간이 소요되었다. 얼굴 검출이 어려운 경우에는 12초 이상 소요되거나 일부 타임아웃이 발생하였다. 이를 통해 본 시스템의 주요 병목은 이미지 전송이나 YOLO 탐지보다 DeepFace 얼굴 검출 및 임베딩 비교 단계에 있음을 확인하였다.

9.5 해상도별 처리 속도 및 판별 안정성 비교

해상도	처리 속도	얼굴 판별 안정성	최종 적용 여부
1280×720	처리 가능(6~7초)	얼굴 영역 확보 가능	비교 실험용
1920×1080	큰 속도 저하 없음(6~8초)	얼굴 영역이 더 선명함	최종 적용

1280×720과 1920×1080 해상도를 비교한 결과, 전체 처리 시간의 대부분은 DeepFace 얼굴 검출 및 임베딩 비교에서 발생하였다. 따라서 해상도 증가가 전체 처리 시간에 미치는 영향은 예상보다 크지 않았다. 반면 1920×1080 해상도에서는 얼굴 영역이 더 선명하게 확보되어 얼굴 검출과 distance 계산 안정성 측면에서 유리하였다. 이에 따라 최종 실험에서는 1920×1080 해상도를 적용하였다.

9.6 실험 결과 종합

구분	확인 결과
라즈베리파이 실시간 촬영	USB 웹캠을 통한 주기적 이미지 캡처 확인
서버 전송	HTTP POST 방식으로 /detect API 전송 확인
사람 탐지	YOLOv8n 기반 person 탐지 확인
등록 인물 판별	일반 정면 조건에서 Registered 판별 확인
미등록 인물 판별	Unknown 또는 Unidentified 판별 확인
이벤트 매핑	final_result에 따라 event_type 자동 분류 확인
알림 기능	Unknown, Unidentified 발생 시 Telegram·Email 알림 확인
결과 저장	MySQL 및 alert_log.txt 저장 확인
애매한 인물 판별	애매한 distance 구간을 Unidentified로 처리

10. 최종 보완 사항 및 보안 설정

구분	7주차	최종 주차 보완
입력 방식	저장된 이미지 기반 테스트	라즈베리파이 USB 웹캠 실시간 촬영
서버 구조	YOLO/DeepFace 분석 코드 중심	FastAPI /detect, /health API 기반 서버
얼굴 판별	0.35/0.5 threshold 적용	0.30 / 0.47 2단계 threshold 적용
판별 결과	Registered / Unknown / Unidentified	Unidentified 구간 명확화
Detector	RetinaFace + OpenCV 검토	RetinaFace 단일 적용으로 안정성 우선
Early Exit	처리 속도 개선용 적용 검토	실시간 안정성 확보를 위해 비활성화
알림 구조	텔레그램·이메일 구현	threading 기반 비동기 전송으로 응답 지연 완화
민감 정보	코드 직접 입력 가능성	.env 파일로 분리하고 .gitignore로 GitHub 업로드 방지
기능 제어	코드 수정으로 알림/DB 기능 변경	환경변수로 DB, Telegram, Email ON/OFF 제어
해상도	1280x720	1920×1080 적용
성능 확인	정적 이미지 결과 확인	라즈베리파이 , 서버 , DB, 알림 동시 검증

10.1 .env 기반 민감 정보 분리

기존 코드에서는 MySQL 접속 비밀번호, Telegram Bot Token, Telegram Chat ID, Gmail 앱 비밀번호와 같은 민감 정보가 server_realtime.py 내부에 직접 작성될 수 있었다. 이러한 방식은 GitHub와 같은 외부 저장소에 코드가 업로드될 경우 민감 정보가 노출될 위험이 있다. 이를 개선하기 위해 python-dotenv 라이브러리를 사용하여 DB 접속 정보, Telegram 설정, Gmail SMTP 설정, 기능 활성화 여부를 .env 파일로 분리하였다.

[서버 코드] - .env 파일에서 값을 읽어오는 부분

```
import os
from dotenv import load_dotenv
load_dotenv()
# MySQL DB 설정
DB_CONFIG = {
    "host": os.getenv("DB_HOST", "127.0.0.1"),
    "port": int(os.getenv("DB_PORT", "3306")),
    "user": os.getenv("DB_USER", "root"),
    "password": os.getenv("DB_PASSWORD", ""),
    "database": os.getenv("DB_NAME", "capstone_db"),
    "charset": "utf8mb4",
}
}# Telegram 설정
TELEGRAM_BOT_TOKEN = os.getenv("TELEGRAM_BOT_TOKEN", "")
TELEGRAM_CHAT_ID = os.getenv("TELEGRAM_CHAT_ID", "")
# Gmail SMTP 설정
SMTP_SERVER = os.getenv("SMTP_SERVER", "smtp.gmail.com")
SMTP_PORT = int(os.getenv("SMTP_PORT", "587"))
EMAIL_SENDER = os.getenv("EMAIL_SENDER", "")
EMAIL_APP_PASSWORD = os.getenv("EMAIL_APP_PASSWORD", "")
EMAIL_RECEIVER = os.getenv("EMAIL_RECEIVER", "")
# 기능 ON/OFF 설정
ENABLE_DB_SAVE = os.getenv("ENABLE_DB_SAVE", "True") == "True"
ENABLE_TELEGRAM_ALERT = os.getenv("ENABLE_TELEGRAM_ALERT", "True") == "True"
ENABLE_EMAIL_ALERT = os.getenv("ENABLE_EMAIL_ALERT", "True") == "True"
```

[.env파일] - 실제 설정값을 저장하는 부분

```
# MySQL DB
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=root
DB_PASSWORD=*****
DB_NAME=capstone_db
# Telegram Bot
TELEGRAM_BOT_TOKEN=*****
TELEGRAM_CHAT_ID=*****
# Gmail SMTP
SMTP_SERVER=smtp.gmail.com
SMTP_PORT=587
EMAIL_SENDER=your_email@gmail.com
EMAIL_APP_PASSWORD=*****
EMAIL_RECEIVER=receiver_email@gmail.com
# Feature toggles
ENABLE_DB_SAVE=True
ENABLE_TELEGRAM_ALERT=True
ENABLE_EMAIL_ALERT=True
```

10.2 .gitignore를 통한 민감 파일 및 결과 파일 제외

.env 파일에는 DB 비밀번호, Telegram Bot Token, Gmail 앱 비밀번호와 같은 민감 정보가 포함되므로 GitHub에 업로드되지 않도록 .gitignore에 등록하였다. 또한 실행 결과 이미지, 업로드 이미지, 모델 파일, 가상환경 폴더 등 용량이 크거나 재생성 가능한 파일도 함께 제외하였다. 이를 통해 저장소에는 핵심 소스 코드만 관리하고, 민감 정보와 대용량 결과 파일은 로컬 환경에서만 관리하도록 구성하였다.

```
# Sensitive
.env
*.env
# Python
venv/
__pycache__/
*.pyc
*.pyo
# Large outputs
```

results/
uploads/
*.zip
Model files
*.pt
IDE
.vscode/
.idea/
*.swp
OS
.DS_Store
Thumbs.db
Logs
*.log

11. 결론

이번 최종 구현에서는 라즈베리파이 USB 웹캠과 PC FastAPI 서버를 연동하여 실시간 인물 감지 및 등록·미등록 인물 판별 시스템을 완성하였다. 라즈베리파이는 1920×1080 해상도로 이미지를 촬영하여 서버로 전송하고, 서버는 YOLOv8n을 이용해 사람을 탐지한 뒤 DeepFace ArcFace 모델을 통해 등록 얼굴과 비교하였다.

실시간 테스트 결과 등록 인물은 일반적인 정면 상태에서 Registered로 정상 판별되었고, 미등록 인물은 Unknown 또는 Unidentified로 분류되어 텔레그램 및 이메일 알림이 전송되었다. 특히 등록 인물과 미등록 인물의 distance 값이 일부 겹치는 구간을 고려하여, 0.30 미만은 Registered, 0.30 이상 0.47 미만은 Unidentified, 0.47 이상은 Unknown으로 분류하는 2단계 threshold 구조를 적용하였다.

다만 DeepFace 기반 얼굴 분석으로 인해 사람이 감지된 경우 수 초 이상의 처리 시간이 소요되었고, 표정 변화나 촬영 조건에 따라 Unidentified로 분류되는 사례가 발생하였다. 향후에는 다양한 표정과 각도의 등록 얼굴 데이터를 추가하고, GPU 기반 추론 및 다중 프레임 누적 판단 구조를 적용하여 처리 속도와 판별 안정성을 개선할 필요가 있다.